LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI STRUKTUR DATA *QUEUE* MENGGUNAKAN *ARRAY* DAN *LINKEDLIST* DALAM JAVA

Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 3 dengan judul "Implementasi Struktur Data *Queue* Menggunakan *Array* dan *Linkedlist* dalam *Java*" tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi salah satu tugas dalam mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep struktur data *Queue* serta penerapannya dalam pemrograman Java. Melalui praktikum ini, penulis mempelajari bagaimana mengelola data menggunakan prinsip *FIFO (First In First Out)* dengan menerapkan konsep *ADT*, serta mengimplementasikan operasi di dalamnya. Selain itu, penulis juga mengimplementasikan *Queue* menggunakan *Array* dan *LinkedList*, serta menerapkannya dalam beberapa studi kasus seperti iterasi elemen dalam *Queue* dan proses *reverse* data menggunakan *Stack*.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 28 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ii

DAFTAR ISI iii

BAB I PENDAHULUAN 5

1.1 Latar Belakang 5

1.2 Tujuan 6

1.3 Manfaat 6

BAB II PEMBAHASAN 7

2.1 Dasar Teori 7

2.1.1* Konsep Kerja *Queue (FIFO) 7

2.1.2 Operasi Dasar *Queue* 8

2.2 Langkah Kerja 9

2.2.1 Pembuatan *Package* dan *Class* 9

2.2.2 Membuat Program *QueueArray*_2511532002 13

2.2.3 Membuat Program *QueueArrayDriver*_2511532002 16

2.2.4 Membuat Program *QueueLinkedList*_2511532002 18

2.2.5 Membuat Program *ReverseData_2511532002* 19

2.2.6 Membuat Program *IterasiQueue_2511532002* 20

2.2.7 *Upload* ke GitHub 21

2.3 Hasil 24

2.3.1 Hasil *QueuekArray*_2511532002 24

2.3.2 Hasil *QueuekArrayDriver*_2511532002 25

2.3.3* Hasil *QueueLinkedList_2511532002 25

2.3.4* Hasil *ReverseData_2511532002 25

2.3.5* Hasil *IterasiQueue_2511532002 26

BAB III KESIMPULAN **27**

3.1 Kesimpulan **27**

3.2 Saran **27**

DAFTAR PUSTAKA **28**

TUGAS PRAKTIKUM PEKAN 3 ..... **29**

1. Kode Program **29**

1.1 Kelas *Queue_2511532002* **29**

1.2 Kelas Driver AntrianLoket_2511532002 ..... **32**

2. Hasil **35**

3. Kesimpulan **36**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pengelolaan data yang efisien dan terstruktur dalam proses komputasi. Salah satu aspek penting dalam pemrograman adalah penggunaan struktur data yang tepat untuk mengatur aliran data sesuai dengan kebutuhan sistem. Salah satu struktur data yang banyak digunakan adalah *Queue* (antrian), yang berperan dalam mengelola data secara berurutan dan sistematis.

Queue merupakan struktur data linear yang menerapkan prinsip *FIFO (First In First Out*), yaitu elemen yang pertama kali masuk akan menjadi elemen pertama yang keluar. Konsep ini banyak digunakan dalam berbagai kasus nyata, seperti sistem antrian, manajemen proses pada sistem operasi, serta pengolahan data secara berurutan dalam aplikasi.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar *Queue*, cara implementasinya dalam bahasa *Java*, serta penerapannya dalam berbagai kasus pemrograman. Dengan pemahaman tersebut, penulis diharapkan mampu memilih dan menggunakan struktur data yang tepat untuk menyelesaikan permasalahan secara efektif dan efisien.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1.2.1 Memahami konsep dasar struktur data *Queue* dalam *Abstract Data Type (ADT)*.

1.2.2 Mampu mengimplementasikan struktur data *Queue* menggunakan *Array* dan *LinkedList* dalam bahasa pemrograman *Java.*

1.2.3 Mampu mengimplementasikan operasi dasar *Queue* seperti *enqueue, dequeue, front,* dan *rear.*

1.2.4 Mampu menerapkan menerapkan *Queue* dalam penyelesaian masalah seperti iterasi data dan proses *reverse* menggunakan *Stack.*

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

1.3.1 Menambah pemahaman mengenai konsep *ADT Queue* dalam pemrograman.

1.3.2 Meningkatkan kemampuan dalam mengimplementasikan struktur data *Queue* menggunakan berbagai methode.

1.3.3 Membantu dalam memahami penerapan *Queue* dalam pengelolaan data secara berurutan (*FIFO*).

1.3.4 Menjadi dasar dalam pengembangan program yang lebih kompleks di masa yang akan datang.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Queue atau antrian merupakan salah satu struktur data linear yang digunakan untuk menyimpan sekumpulan data dengan mekanisme tertentu. Dalam struktur data ini, penambahan elemen dilakukan di bagian belakang (*rear*), sedangkan penghapusan elemen dilakukan di bagian depan (*front*).

Queue termasuk dalam kategori *Abstract Data Type (ADT)* yang mendefinisikan operasi-operasi yang dapat dilakukan tanpa harus mengetahui detail implementasinya. Struktur data ini banyak digunakan dalam berbagai aplikasi, seperti sistem antrian, manajemen proses pada sistem operasi, serta pengolahan data secara berurutan.

Dalam bahasa pemrograman *Java, Queue* direpresentasikan sebagai sebuah *interface* dalam *Java Collection Framework* yang menyediakan metode untuk menambahkan, menghapus, dan mengakses elemen. *Queue* tidak dapat dibuat secara langsung, melainkan harus menggunakan kelas implementasi seperti *LinkedList*.

2.1.1 Konsep Kerja *Queue (FIFO)*

Queue bekerja dengan prinsip *FIFO (First In First Out*), yaitu elemen yang pertama kali masuk akan menjadi elemen pertama yang keluar. Konsep ini menyerupai antrian di kehidupan sehari-hari, seperti antrian di kasir atau loket pelayanan, di mana orang yang datang lebih dulu akan dilayani lebih dahulu.

2.1.2 Operasi Dasar *Queue*

Berikut merupakan beberapa operasi dasar yang terdapat pada *Queue* antara lain:

1. *Enqueue*

   Operasi untuk menambahkan elemen ke dalam *queue* pada posisi belakang (*rear*).
2. *Dequeue*

   Operasi untuk menghapus elemen dari bagian depan (*front*).
3. *Front / Peek*

   Operasi untuk melihat elemen terdepan tanpa menghapusnya.
4. *Rear*

   Operasi untuk melihat elemen terakhir dalam *queue*.
5. *isEmpty*

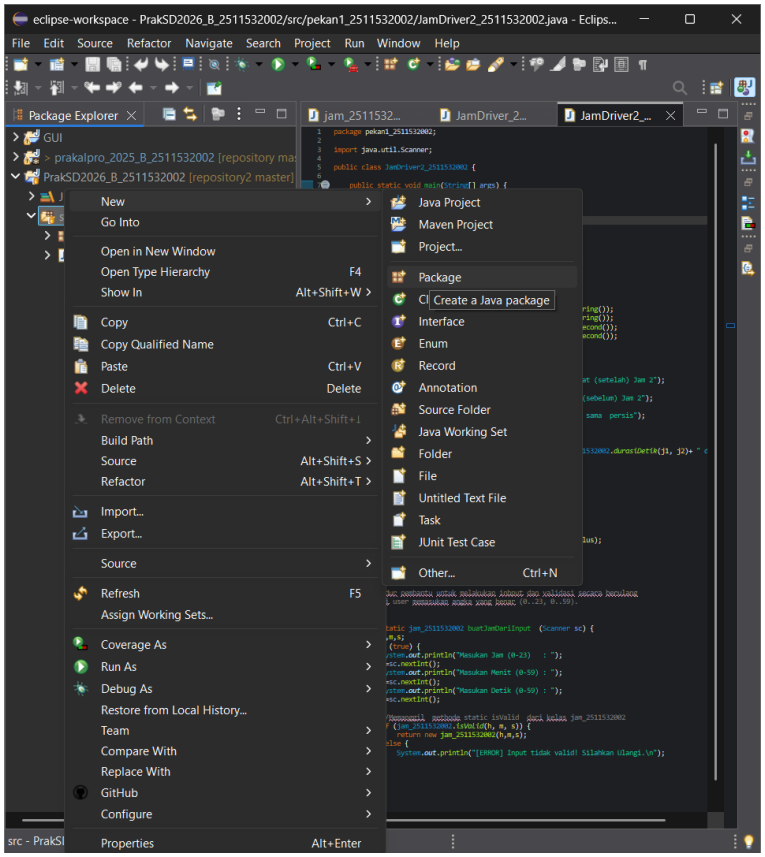
   Digunakan untuk mengecek apakah queue dalam keadaan kosong.
6. *isFull* (pada implementasi array)

   Digunakan untuk mengecek apakah *queue* sudah pen

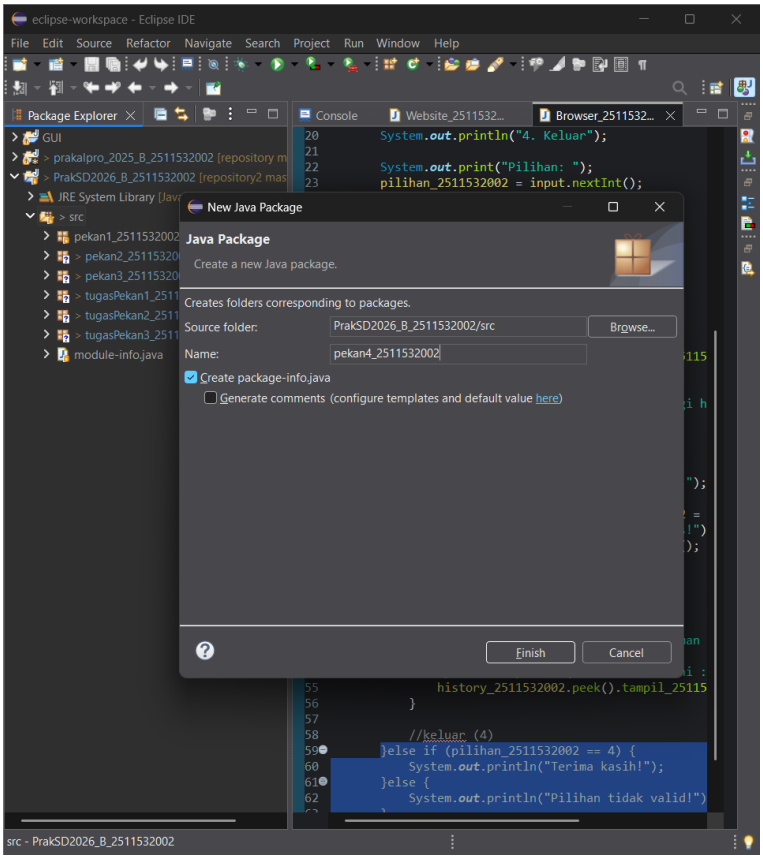
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

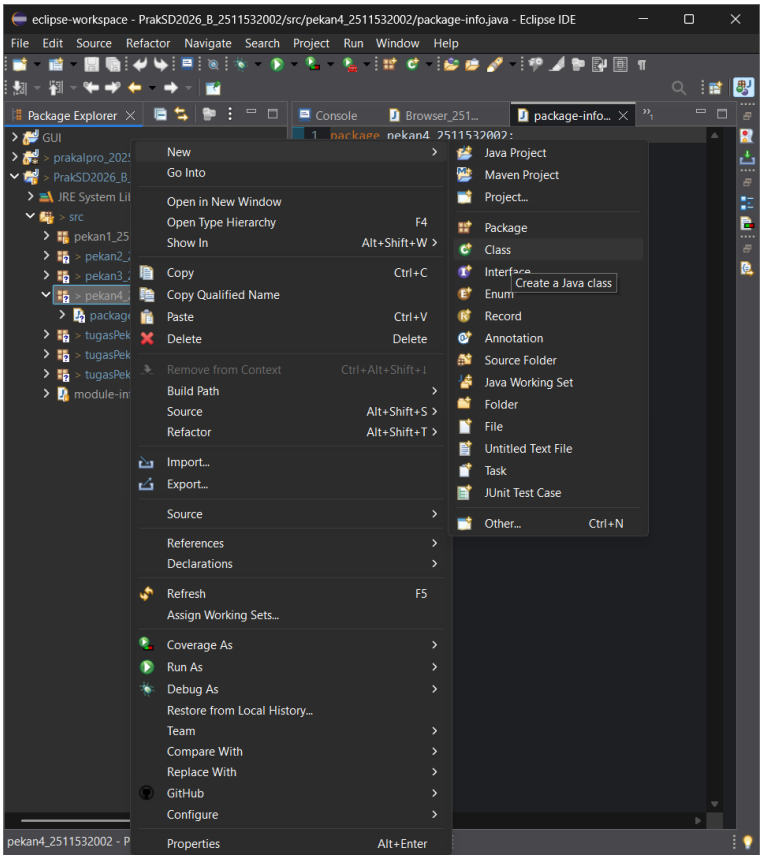
1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* *"PrakSD2026_B_2511532002"* lalu pilih *"New"* dan klik *"Package"*.



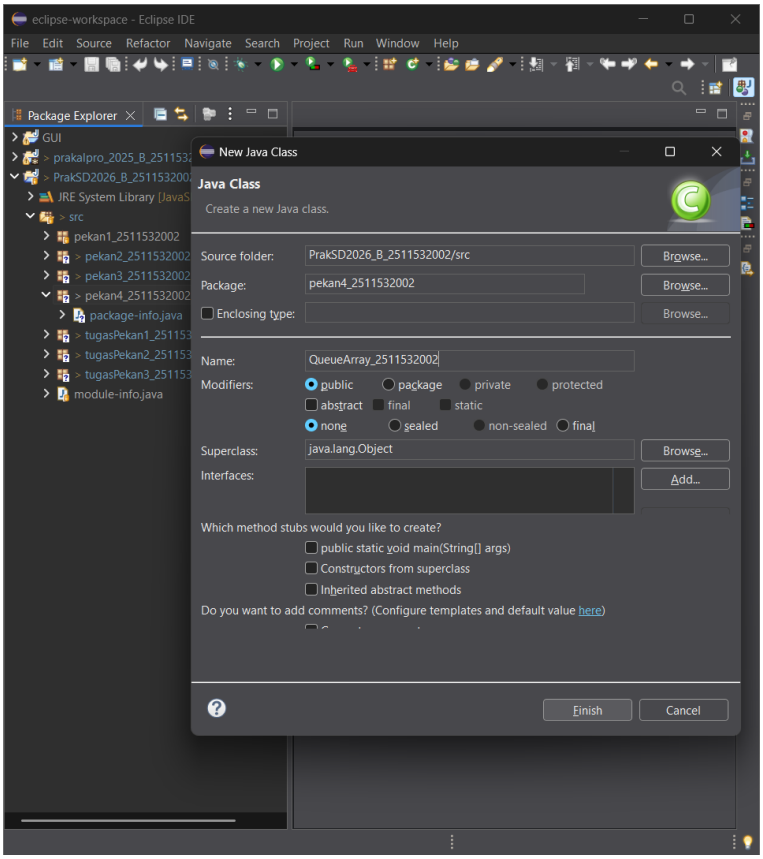
2) Setelah itu akan muncul *pop up* "*Java Package*", buat nama *package* dengan ketentuan tanpa *capslock, space*, dan karakter khusus lainnya. Lalu klik "*finnish*".



3) Selajutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih "*New*" dan klik "*Class*".



4) Setelah itu pada menu "*Java Class*" buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi.



2.2.2 Membuat Program *QueueArray*_2511532002

QueueArray_2511532002.java

```java
package pekan4_2511532002;

public class QueueArray_2511532002 {
    int front_2002, rear_2002, size_2002;
    int capacity_2002;
    int array_2002[];

    public QueueArray_2511532002(int capacity_2002) {
        this.capacity_2002=capacity_2002;
        front_2002=this.size_2002=0;
        rear_2002=capacity_2002-1;
        array_2002=new int [this.capacity_2002];
    }

    boolean isFull (QueueArray_2511532002 queue) {
        return (queue.size_2002 == queue.capacity_2002);
    }

    boolean isEmpty (QueueArray_2511532002 queue) {
        return (queue.size_2002 == 0);
    }

    void enqueue (int item) {
        if (isFull (this))
            return;
        this.rear_2002=(this.rear_2002 +1)& this.capacity_2002;
        this.array_2002[this.rear_2002]=item;
        this.size_2002 = this.size_2002+1;
        System.out.println(item + "enqueue to queue");
    }

    int dequeue () {
        if (isEmpty (this))
            return Integer.MIN_VALUE;
        int item =this.array_2002[this.front_2002];
        this.front_2002=(this.front_2002+1)%this.capacity_2002;
        this.size_2002=this.size_2002-1;
        return item;
    }

    int front_2002() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;

        return this.array_2002[this.front_2002];
    }

    int rear_2002() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;
        return this.array_2002[this.rear_2002];
    }

    //Mencetak elemen antrian
    void display_2002() {
        int i;
        if (front_2002 == rear_2002) {
            System.out.println("\nAntrian Kosong\n");
            return;
        }

        //Kunjungi dari belakang, dan cetak
        for (i=front_2002(); i<rear_2002; i++) {
            System.out.printf("%d <---", array_2002 [i]);
        }
        return;
        }
}
```

Setelah membuat *class QueueArray_2511532002,* maka mahasiswa dapat menulis *syntax* nya.

1. Menentukan atribut (*Type*) yang digunakan dalam *Queue*.
   int front_2002, rear_2002, size_2002;
   int capacity_2002;
   int array_2002[];

2. Membuat konstruktor untuk menginisialisasi *Queue*.

```
public QueueArray_2511532002(int capacity_2002) {
    this.capacity_2002=capacity_2002;
    front_2002=this.size_2002=0;
    rear_2002=capacity_2002-1;
    array_2002=new int [this.capacity_2002];
}
```

3. Membuat *methode* untuk mengecek apakah *Queue penuh (isFull)*.

```
boolean isFull_2002 (QueueArray_2511532002 queue_2002) {
    return (queue_2002.size_2002 == queue_2002.capacity_2002);
}
```

4. Membuat *methode* untuk mengecek apakah *Queue* kosong *(isEmpty)*.

```
boolean isEmpty_2002(QueueArray_2511532002 queue_2002) {
    return (queue_2002.size_2002 == 0);
}
```

5. Membuat *methode* untuk menambahkan *data ke dalam Queue (enqueue)*.

```
void enqueue_2002(int item) {
    if (isFull_2002 (this))
        return;
    this.rear_2002=(this.rear_2002 +1)% this.capacity_2002;
    this.array_2002[this.rear_2002]=item;
    this.size_2002 = this.size_2002+1;
    System.out.println(item + "enqueue to queue");
}
```

6. Membuat *methode* untuk menghapus data dari *Queue (dequeue)*

```
int dequeue_2002 () {
    if (isEmpty_2002 (this))
        return Integer.MIN_VALUE;
    int item =this.array_2002[this.front_2002];
    this.front_2002=(this.front_2002+1)%this.capacity_2002;
    this.size_2002=this.size_2002-1;
    return item;
}
```

7. Membuat *methode untuk melihat elemen terdepan (front)*.

```
int front_2002() {
    if (isEmpty_2002(this))
        return Integer.MIN_VALUE;
    return this.array_2002[this.front_2002];
}
```

8. Membuat methode untuk melihat elemen terakhir (*rear*).

```
int rear_2002() {
    if (isEmpty_2002(this))
        return Integer.MIN_VALUE;
    return this.array_2002[this.rear_2002];
}
```

9. Membuat methode untuk menampilkan isi *Queue*.

```
void display_2002() {
    int i;
    if (front_2002 == rear_2002) {
        System.out.println("\nAntrian Kosong\n");
        return;
    }

    for (i=front_2002; i<rear_2002; i++) {
        System.out.printf("%d <---", array_2002 [i]);
    }
    return;
}
```

2.2.3 Membuat Program *QueueArrayDriver_*2511532002

```java
package pekan4_2511532002;

public class QueueArrayDriver_2511532002 {

    public static void main(String[] args) {
        QueueArray_2511532002 queue_2002 =new QueueArray_2511532002(1000);
        queue_2002.enqueue_2002(10);
        queue_2002.enqueue_2002(20);
        queue_2002.enqueue_2002(30);
        queue_2002.enqueue_2002(40);

        System.out.println("Item di depan "+ queue_2002.front_2002());
        System.out.println("Item paling belakang "+queue_2002.rear_2002());
        System.out.println("\nTampilan Queue");
        queue_2002.display_2002();
        System.out.println();

        System.out.println();
        System.out.println(queue_2002.dequeue_2002() +" dihapus dari Queue");
        System.out.println("item di depan    : "+queue_2002.front_2002());
        System.out.println("item di belakang : "+queue_2002.rear_2002());
        System.out.println("Tampilan Queue setelah satu Data dihapus");
        queue_2002.display_2002();
    }
}
```

```
<terminated> QueueArrayDriver_2511532002 [Java Application] C:\Users\user\.p2\pool\plugins\org
10enqueue to queue
20enqueue to queue
30enqueue to queue
40enqueue to queue
Item di depan 10
Item paling belakang 40

Tampilan Queue
10 <---20 <---30 <---

10 dihapus dari Queue
item di depan    : 20
item di belakang : 40
Tampilan Queue setelah satu Data dihapus
20 <---30 <---
```

Membuat *class driver* sebagai program utama. *Class* ini digunakan sebagai tempat menjalankan program *Queue.*

1. Membuat *methode* main sebagai titik awal eksekusi program.
   public static void main(String[] args) {

2. Membuat objek dari *class Queue.* Digunakan dalam program.
   QueueArray_2511532002 queue_2002 = new QueueArray_2511532002(1000);

3. Menambahkan data ke dalam *Queue (enqueue).* digunakan untuk memasukkan beberapa elemen ke dalam *Queue.*
   queue_2002.enqueue_2002(10);
   queue_2002.enqueue_2002(20);
   queue_2002.enqueue_2002(30);
   queue_2002.enqueue_2002(40);
   Data yang dimasukkan ke dalam *queue* adalah*:*
   10 \ 20 \ 30 \ 40

4. Menampilkan elemen terdepan dan terakhir. Digunakan untuk melihat elemen paling depan dan terakir..
   System.out.println("Item di depan "+ queue_2002.front_2002());
   System.out.println("Item paling belakang "+queue_2002.rear_2002());

5. Menampilkan seluruh isi *Queue,*
   System.out.println("\nTampilan Queue");
   queue_2002.display_2002();
   System.out.println();

6. Menghapus satu elemen dari *Queue* (*dequeue*).
   System.out.println(queue_2002.dequeue_2002() +" dihapus dari Queue");

7. Menampilkan kondisi Queue setelah penghapusan.
   System.out.println("item di depan : "+queue_2002.front_2002());
   System.out.println("item di belakang : "+queue_2002.rear_2002());

8. Menampilkan isi Queue setelah data dihapus.
   System.out.println("Tampilan Queue setelah satu Data dihapus");
   queue_2002.display_2002();

2.2.4 Membuat Program *QueueLinkedList*_2511532002

```java
package pekan4_2511532002;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511532002 {

    public static void main(String[] args) {
        Queue<Integer> q_2002=new LinkedList<>();

        //tambah elemen ke Antrian
        for (int i=0; i<6; i++)
            q_2002.add(i);

        //Menampilkan isi Antrian
        System.out.println("\nElemen Antrian "+q_2002);

        //Menghapus Head Antrian
        int hapus_2002=q_2002.remove();
        System.out.println("\nHapus Elemen = "+hapus_2002);
        System.out.println(q_2002);

        //Untuk melihat Antrian depan
        int depan_2002=q_2002.peek();
        System.out.println("\nKepala Antrian = "+depan_2002);

        //Menampilkan banyak data
        int banyak_2002= q_2002.size();
        System.out.println("\nSize Antrian = "+ banyak_2002);
    }

}
```

```
Console
<terminated> QueueLinkedList_2511532002 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjdk.hotsp

Elemen Antrian [0, 1, 2, 3, 4, 5]

Hapus Elemen = 0
[1, 2, 3, 4, 5]

Kepala Antrian = 1

Size Antrian = 5
```

1. Membuat *method main* sebagai awal untuk eksekusi program.
   public static void main(String[] args) {

2. Membuat objek *Queue* menggunakan *LinkedList*
   Queue<Integer> q_2002 = new LinkedList<>();

3. Menambahkan elemen ke dalam *Queue* dan menampilkannya.
   for (int i=0; i<6; i++)
   q_2002.add(i);
   System.out.println("\nElemen Antrian "+q_2002);

4. Menghapus elemen pertama (*head*) dari *Queue* dan menampilkan *queue* setelah penghapusan.
   int hapus_2002 = q_2002.remove();
   System.out.println("\nHapus Elemen = "+hapus_2002);
   System.out.println(q_2002);

5. Melihat elemen terdepan tanpa menghapusnya.
   int depan_2002 = q_2002.peek();
   System.out.println("\nKepala Antrian = "+depan_2002);

6. Menampilkan jumlah elemen dalam Queue.
   int banyak_2002 = q_2002.size();
   System.out.println("\nSize Antrian = "+ banyak_2002);

2.2.5 Membuat Program *ReverseData_2511532002*

```java
package pekan4_2511532002;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532002 {

    public static void main(String[] args) {
        Queue<Integer> q_2002 =new LinkedList<Integer>();
        q_2002.add(1);
        q_2002.add(2);
        q_2002.add(3);

        System.out.println("Sebelum reverse "+ q_2002);
        Stack<Integer> s_2002 = new Stack<Integer>();
        while (!q_2002.isEmpty()) {
            s_2002.push(q_2002.remove());
        }
        while (!s_2002.isEmpty()) {
            q_2002.add(s_2002.pop());
        }
        System.out.println("Sesudah reverse = "+q_2002);
        }
}
```

```
Sebelum reverse [1, 2, 3]
Sesudah reverse = [3, 2, 1]
```

1. Membuat *method main* sebagai titik awal eksekusi program.
   public static void main(String[] args) {

2. Membuat objek *Queue* dan menambahkan elemen.
   Queue<Integer> q_2002 = new LinkedList<Integer>();
   q_2002.add(1);
   q_2002.add(2);
   q_2002.add(3);

3. Menampilkan isi *Queue* sebelum reverse.
   System.out.println("Sebelum reverse "+ q_2002);

4. Membuat objek *Stack.*
   Stack<Integer> s_2002 = new Stack<Integer>();

5. *Memindahkan data dari Queue ke Stack.*
   while (!q_2002.isEmpty()) {
   s_2002.push(q_2002.remove());
   }

6. *Memindahkan kembali data dari Stack ke Queue.*
   while (!s_2002.isEmpty()) {
   q_2002.add(s_2002.pop());
   }
7. Menampilkan hasil setelah *reverse.*
   System.out.println("Sesudah reverse = "+q_2002);

2.2.6 Membuat Program *IterasiQueue_2511532002*

```java
package pekan4_2511532002;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511532002 {

    public static void main(String[] args) {
        Queue<String> q_2002=new LinkedList<>();

        q_2002.add("Praktikum");
        q_2002.add("Struktur");
        q_2002.add("Data");
        q_2002.add("Dan");
        q_2002.add("Algoritma");
        Iterator<String> iterator_2002 = q_2002.iterator();
        while (iterator_2002.hasNext()) {
            System.out.print(iterator_2002.next()+ " ");
        }
    }
}
```

```
Praktikum Struktur Data Dan Algoritma
```

1. Membuat *method main* sebagai titik awal eksekusi program.
   public static void main(String[] args) {

2. Membuat objek *Queue* bertipe *String*.
   Queue<String> q_2002 = new LinkedList<>();

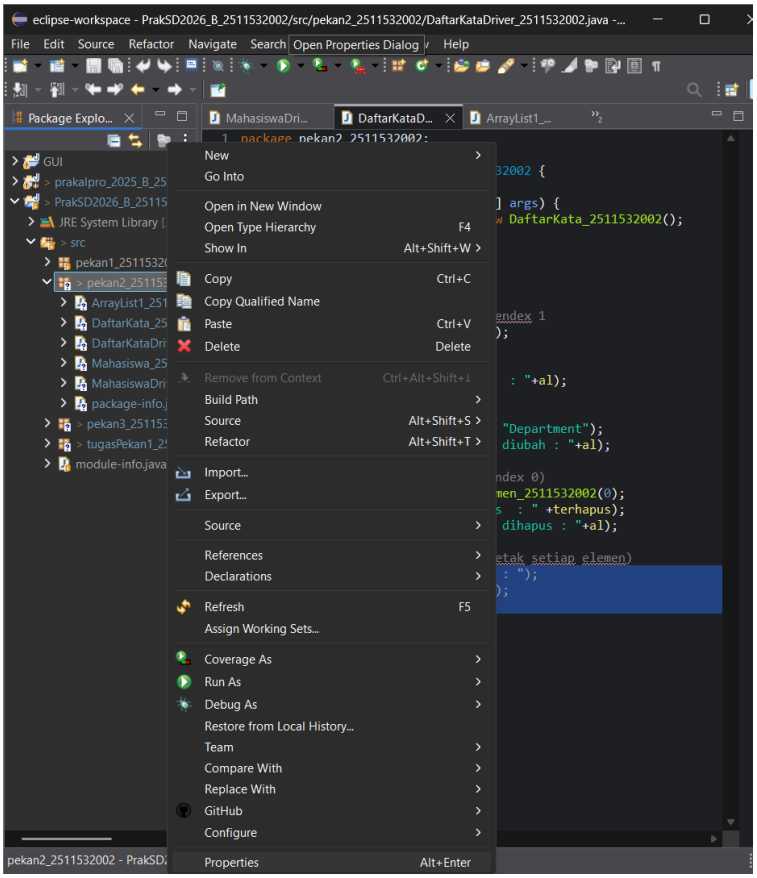
3. Menambahkan elemen ke dalam *Queue*.
   q_2002.add("Praktikum");
   q_2002.add("Struktur");
   q_2002.add("Data");
   q_2002.add("Dan");
   q_2002.add("Algoritma");

4. Membuat objek Iterator untuk mengakses elemen *Queue*.
   Iterator<String> iterator_2002 = q_2002.iterator();

5. Melakukan iterasi untuk menampilkan seluruh elemen *Queue*.
   while (iterator_2002.hasNext()) {
   System.out.print(iterator_2002.next() + " ");
   }

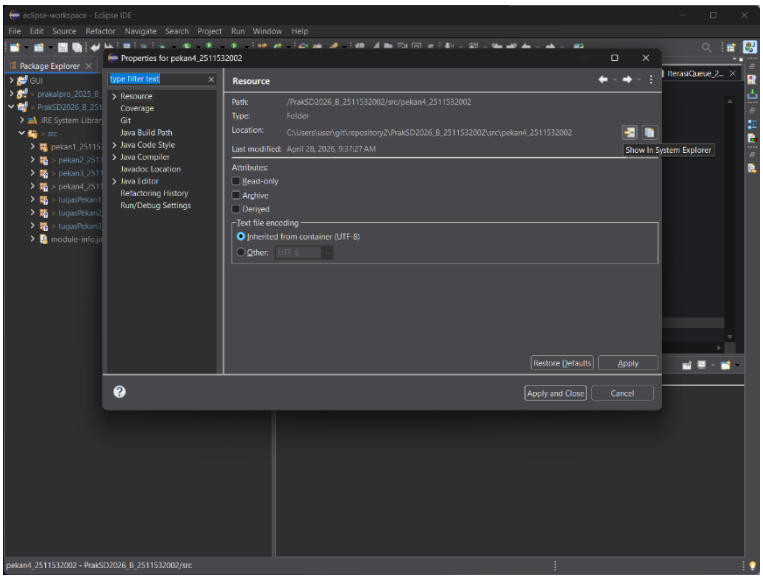
2.2.7 *Upload* ke GitHub

1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukan program yang dibuat di Eclips ke *repository* GitHub. Selain dengan cara *"Team"* atau *"Commit and Push"*.

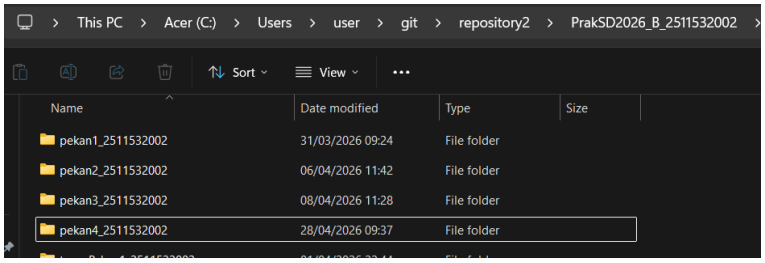


Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package,* lalu pilih *properties.*

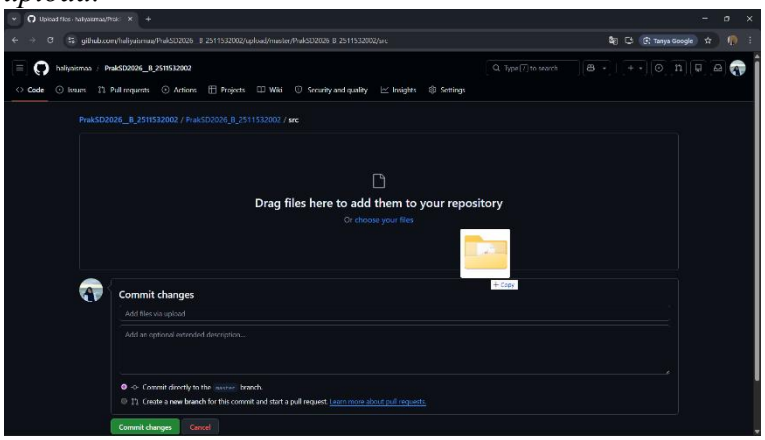
2. Maka akan muncul *pop up menu "Properteis for pekan4_2511532002"*. Lalu klik *icon "Show in System Folder"*.



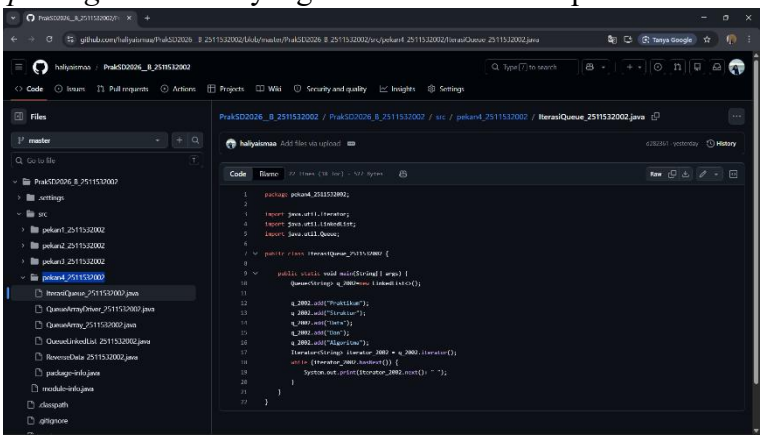
3. Maka akan menunjukan lokasi *file "pekan4_2511532002"* pada *File Explorer computer.*



4. Setelah itu *drag and drop file "pekan4_2511532002"* ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload.*



5. Maka dapat dilihat pada *repository GitHub* bahwa file "*pekan4_2511532002*" telah terunggah via *Upload.*
   Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.



2.3 Hasil

2.3.1 Hasil *QueuekArray*_2511532002

QueueArray_2511532002.java

```java
package pekan4_2511532002;

public class QueueArray_2511532002 {
    int front_2002, rear_2002, size_2002;
    int capacity_2002;
    int array_2002[];

    public QueueArray_2511532002(int capacity_2002) {
        this.capacity_2002=capacity_2002;
        front_2002=this.size_2002=0;
        rear_2002=capacity_2002-1;
        array_2002=new int [this.capacity_2002];
    }

    boolean isFull (QueueArray_2511532002 queue) {
        return (queue.size_2002 == queue.capacity_2002);
    }

    boolean isEmpty (QueueArray_2511532002 queue) {
        return (queue.size_2002 == 0);
    }

    void enqueue (int item) {
        if (isFull (this))
            return;
        this.rear_2002=(this.rear_2002 +1)& this.capacity_2002;
        this.array_2002[this.rear_2002]=item;
        this.size_2002 = this.size_2002+1;
        System.out.println(item + "enqueue to queue");
    }

    int dequeue () {
        if (isEmpty (this))
            return Integer.MIN_VALUE;
        int item =this.array_2002[this.front_2002];
        this.front_2002=(this.front_2002+1)%this.capacity_2002;
        this.size_2002=this.size_2002-1;
        return item;
    }

    int front_2002() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;

        return this.array_2002[this.front_2002];
    }

    int rear_2002() {
        if (isEmpty(this))
            return Integer.MIN_VALUE;
        return this.array_2002[this.rear_2002];
    }

    //Mencetak elemen antrian
    void display_2002() {
        int i;
        if (front_2002 == rear_2002) {
            System.out.println("\nAntrian Kosong\n");
            return;
        }

        //Kunjungi dari belakang, dan cetak
        for (i=front_2002(); i<rear_2002; i++) {
            System.out.printf("%d <---", array_2002 [i]);
        }
        return;
        }
}
```

2.3.2 Hasil *QueuekArrayDriver*_2511532002

```java
package pekan4_2511532002;

public class QueueArrayDriver_2511532002 {

    public static void main(String[] args) {
        QueueArray_2511532002 queue_2002 =new QueueArray_2511532002(1000);
        queue_2002.enqueue_2002(10);
        queue_2002.enqueue_2002(20);
        queue_2002.enqueue_2002(30);
        queue_2002.enqueue_2002(40);

        System.out.println("Item di depan "+ queue_2002.front_2002());
        System.out.println("Item paling belakang "+queue_2002.rear_2002());
        System.out.println("\nTampilan Queue");
        queue_2002.display_2002();
        System.out.println();

        System.out.println();
        System.out.println(queue_2002.dequeue_2002() +" dihapus dari Queue");
        System.out.println("item di depan    : "+queue_2002.front_2002());
        System.out.println("item di belakang : "+queue_2002.rear_2002());
        System.out.println("Tampilan Queue setelah satu Data dihapus");
        queue_2002.display_2002();
    }
}
```

```
<terminated> QueueArrayDriver_2511532002 [Java Application]
10enqueue to queue
20enqueue to queue
30enqueue to queue
40enqueue to queue
Item di depan 10
Item paling belakang 40

Tampilan Queue
10 <---20 <---30 <---

10 dihapus dari Queue
item di depan    : 20
item di belakang : 40
Tampilan Queue setelah satu Data dihapus
20 <---30 <---
```

2.3.3 Hasil *QueueLinkedList_2511532002*

```java
package pekan4_2511532002;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511532002 {

    public static void main(String[] args) {
        Queue<Integer> q_2002=new LinkedList<>();

        //tambah elemen ke Antrian
        for (int i=0; i<6; i++)
            q_2002.add(i);

        //Menampilkan isi Antrian
        System.out.println("\nElemen Antrian "+q_2002);

        //Menghapus Head Antrian
        int hapus_2002=q_2002.remove();
        System.out.println("\nHapus Elemen = "+hapus_2002);
        System.out.println(q_2002);

        //Untuk melihat Antrian depan
        int depan_2002=q_2002.peek();
        System.out.println("\nKepala Antrian = "+depan_2002);

        //Menampilkan banyak data
        int banyak_2002= q_2002.size();
        System.out.println("\nSize Antrian = "+ banyak_2002);
    }

}
```

```
<terminated> QueueLinkedList_2511532002 [Java Application]

Elemen Antrian [0, 1, 2, 3, 4, 5]

Hapus Elemen = 0
[1, 2, 3, 4, 5]

Kepala Antrian = 1

Size Antrian = 5
```

2.3.4 Hasil *ReverseData_2511532002*

```java
package pekan4_2511532002;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532002 {

    public static void main(String[] args) {
        Queue<Integer> q_2002 =new LinkedList<Integer>();
        q_2002.add(1);
        q_2002.add(2);
        q_2002.add(3);

        System.out.println("Sebelum reverse "+ q_2002);
        Stack<Integer> s_2002 = new Stack<Integer>();
        while (!q_2002.isEmpty()) {
            s_2002.push(q_2002.remove());
        }
        while (!s_2002.isEmpty()) {
            q_2002.add(s_2002.pop());
        }
        System.out.println("Sesudah reverse = "+q_2002);
        }
}
```

```
<terminated> ReverseData_2511532002 [Java Application]
Sebelum reverse [1, 2, 3]
Sesudah reverse = [3, 2, 1]
```

2.3.5 Hasil *IterasiQueue_2511532002*

```java
package pekan4_2511532002;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511532002 {

    public static void main(String[] args) {
        Queue<String> q_2002=new LinkedList<>();

        q_2002.add("Praktikum");
        q_2002.add("Struktur");
        q_2002.add("Data");
        q_2002.add("Dan");
        q_2002.add("Algoritma");
        Iterator<String> iterator_2002 = q_2002.iterator();
        while (iterator_2002.hasNext()) {
            System.out.print(iterator_2002.next()+ " ");
        }
    }
}
```

```
Praktikum Struktur Data Dan Algoritma
```

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data *Queue* merupakan salah satu struktur data linear yang bekerja dengan prinsip *FIFO (First In First Out*), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar.

Dalam praktikum ini, *Queue* berhasil diimplementasikan menggunakan dua pendekatan, yaitu *Array (circular queue)* dan *LinkedList*. Selain itu, operasi dasar *Queue* seperti *enqueue, dequeue*, *front*, dan *rear* dapat dijalankan dengan baik sesuai konsep yang ada. Queue juga dapat dikombinasikan dengan struktur data lain, seperti *Stack*, untuk melakukan operasi tertentu seperti membalik urutan data (*reverse*).

Melalui praktikum ini, penulis dapat memahami konsep dasar *Queue*, cara implementasinya dalam bahasa pemrograman *Java*, serta penerapannya dalam berbagai kasus seperti iterasi data dan manipulasi urutan elemen. Dengan demikian, *Queue* menjadi salah satu struktur data yang penting dalam pengolahan data secara terstruktur dan efisien.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami penulisan *syntax* pemrograman *Java* dan penggunaan struktur data untuk menghindari kesalahan atau *error*, pemanggilan *metShod*, serta pengelolaan data dalam program.

DAFTAR PUSTAKA

[1] GeeksforGeeks, “Queue Interface in Java,” 2024. [Online]. Available: https://www.geeksforgeeks.org/java/queue-interface-java/

[2] Oracle, “Interface Queue<E> (Java Platform SE 8),” 2014. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html.

TUGAS PRAKTIKUM PEKAN 3

1. Kode Program

1.1 Kelas *Queue_2511532002*

*Queue_2511532002.java *AntrianLoket_2511532002.java

PrakSD2026_B_2511532002 > src > tugasPekan4_2511532002

```java
package tugasPekan4_2511532002;

public class Queue_2511532002 {
    String queue_2002[];
    int front_2002;
    int rear_2002;
    int max_2002;

    public Queue_2511532002(int size) {
        max_2002 =size;
        queue_2002 = new String[max_2002];
        front_2002 = 0;
        rear_2002 = -1;
    }

    public boolean isEmpty_2002() {
        return rear_2002 == -1;
    }

    public boolean isFull_2002() {
        return rear_2002 == max_2002 -1;
    }

    public void enqueue_2002(String data) {
        if (isFull_2002()) {
            System.out.println("Antrian penuh!");
        }else {
            rear_2002++;
            queue_2002[rear_2002] = data;
            System.out.println("Data berhasil ditambahkan ke antrian");
        }
    }

    public void dequeue_2002() {
        if (isEmpty_2002()) {
            System.out.println("Antrian Kosong!");
        }else {
            String keluar_2002 = queue_2002[front_2002];
            System.out.println(keluar_2002+" telah dilayani.");

            for (int i = front_2002; i<rear_2002; i++) {
                queue_2002[i] = queue_2002[i+1];
            }
            rear_2002--;
        }
    }

    public void tampil_2002() {
        if (isEmpty_2002()) {
            System.out.println("Antrian Kosong!");
        }else {
            System.out.println("Isi Antrian : ");
            for (int i=front_2002; i<=rear_2002; i++) {
                System.out.println((i+1) + ". "+queue_2002[i]);
            }
        }
    }

    public void reverse() {
        if (isEmpty_2002()) {
            System.out.println("Antrian Kosong!");
        }else {
            int i = front_2002;
            int j = rear_2002;

            while (i < j) {
                String temp = queue_2002[i];
                queue_2002[i] = queue_2002[j];
                queue_2002[j] = temp;
                i++;
                j--;
            }
            System.out.println("Antrian berhasil dibalik!");
        }
    }
}
```

Penjelasan :

1) *package tugasPekan4_2511532002;*
   Menjelaskan posisi file berada pada package bernama "*tugaspekan4_2511532002*".

2) *public class Queue_2511532002 {*
   Mendefinisikan *class* bernama *Queue_2511532002* yang digunakan sebagai *Abstract Data Type (ADT*) untuk merepresentasikan struktur data *Queue* (antrian) menggunakan *array*.

3) *Atribut*
   - *String queue[];*
   - *int front_2002;*
   - *int rear_2002;*
   - *int max_2002;*

   Merupakan atribut dari *class Queue_2511532002* yang digunakan untuk menyimpan data seperti.

4) Konstruktor
```
public Queue_2511532002(int size) {
   max_2002 = size;
   queue_2002= new String[max];
   front_2002= 0;
   rear_2002 = -1;
}
```
   Merupakan konstruktor yang digyunakan untuk mengisialisasikan nilai atribut.

5) *Methode isEmpty_2002()*
```
public boolean isEmpty_2002 () {
return rear_2002 == -1;
}
```
   Digunakan untuk mengecek apakah antrian kosong.

6) *Methode isFull_2002 ()*
```
public boolean isFull_2002 () {
   return rear_2002 == max_2002 -1;
}
```
   Digunakan untuk mengecek apakah antrian sudah penuh.

7) *Methode enqueue_2002 ()*

```
public void enqueue_2002 (String data) {
   if (isFull_2002 ()) {
      System.out.println("Antrian penuh!");
   } else {
      rear_2002++;
      queue_2002 [rear_2002] = data;
      System.out.println("Data berhasil ditambahkan ke antrian");
   }
}
```

Digunakan untuk menambahkan data ke dalam antrian (*enqueue*).

8) *Methode dequeue_2002 ()*

```
public void dequeue_2002 () {
   if (isEmpty_2002 ()) {
      System.out.println("Antrian Kosong!");
   } else {
      String keluar_2002 = queue_2002 [front];
      System.out.println(keluar_2002+" telah dilayani.");

      for (int i = front; i<rear; i++) {
         queue[i] = queue[i+1];
      }
      rear_2002--;
   }
}
```

Digunakan untuk menghapus data dari antrian (*dequeue*). Mengecek antrian kosong atau tidak, dan jik tidak baru menghapus data paling depan.

9) *Methode tampil_2002 ()*

```
public void tampil_2002 () {
   if (isEmpty_2002 ()) {
      System.out.println("Antrian Kosong!");
   } else {
      System.out.println("Isi Antrian : ");
      for (int i=fron_2002t; i<=rear_2002; i++) {
         System.out.println((i+1) + ". "+queue_2002 [i]);
      }
   }
}
```

Digunakan untuk menampilkan seluruh isi antrian. Menampilkan data dari posisi front sampai rear.

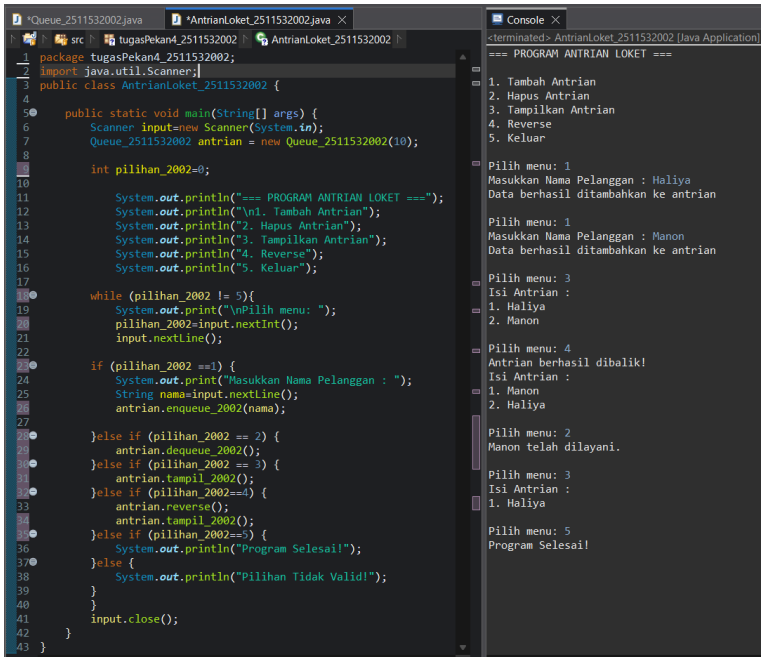
10) *Methode reverse()*

```
public void reverse() {
    if (isEmpty_2002 ()) {
        System.out.println("Antrian Kosong!");
    } else {
        int i = front_2002;
        int j = rear_2002;

        while (i < j) {
            String temp = queu_2002e[i];
            queue_2002 [i] = queue_2002 [j];
            queue_2002 [j] = temp;
            i++;
            j--;
        }
        System.out.println("Antrian berhasil dibalik!");
    }
}
```

Digunakan untuk membalik urutan antrian.

1.2 Kelas Driver AntrianLoket_2511532002



```
package tugasPekan4_2511532002;
import java.util.Scanner;
public class AntrianLoket_2511532002 {

    public static void main(String[] args) {
        Scanner input=new Scanner(System.in);
        Queue_2511532002 antrian = new Queue_2511532002(10);

        int pilihan_2002=0;

            System.out.println("=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("\n1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");

        while (pilihan_2002 != 5){
            System.out.print("\nPilih menu: ");
            pilihan_2002=input.nextInt();
            input.nextLine();

        if (pilihan_2002 ==1) {
            System.out.print("Masukkan Nama Pelanggan : ");
            String nama=input.nextLine();
            antrian.enqueue_2002(nama);

        }else if (pilihan_2002 == 2) {
            antrian.dequeue_2002();
        }else if (pilihan_2002 == 3) {
            antrian.tampil_2002();
        }else if (pilihan_2002==4) {
            antrian.reverse();
            antrian.tampil_2002();
        }else if (pilihan_2002==5) {
            System.out.println("Program Selesai!");
        }else {
            System.out.println("Pilihan Tidak Valid!");
        }
        }
        input.close();
    }
}
```

```
<terminated> AntrianLoket_2511532002 [Java Application]
=== PROGRAM ANTRIAN LOKET ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 1
Masukkan Nama Pelanggan : Haliya
Data berhasil ditambahkan ke antrian

Pilih menu: 1
Masukkan Nama Pelanggan : Manon
Data berhasil ditambahkan ke antrian

Pilih menu: 3
Isi Antrian :
1. Haliya
2. Manon

Pilih menu: 4
Antrian berhasil dibalik!
Isi Antrian :
1. Manon
2. Haliya

Pilih menu: 2
Manon telah dilayani.

Pilih menu: 3
Isi Antrian :
1. Haliya

Pilih menu: 5
Program Selesai!
```

Penjelasan:

1) *package tugasPekan4_2511532002;*
   Menjelaskan posisi file berada pada package bernama "*tugaspekan4_2511532002*".

2) *public class AntrianLoket_2511532002 {*
   Merupakan *class* utama (*driver*) yang digunakan untuk menjalankan program.

3) *public static void main(String[] args) {*
   Merupakan *method* utama yang pertama kali dijalankan saat program dieksekusi.

4) *Scanner input =new Scanner (System.in);*
   Untuk menerima *input* dari *user*.

5) *Queue_2511532002 antrian = new Queue_2511532002(10);*
   Digunakan untuk Membuat objek bernama antrian dari *class Queue* dengan kapasitas maksimal 10 data.

6) *int pilihan_2002 = 0;*
   Sebagai variabel untuk menyimpan pilihan menu dari *user*.

7) {System.out.println("=== PROGRAM ANTRIAN LOKET ===");
   System.out.println("\n1. Tambah Antrian");
   System.out.println("2. Hapus Antrian");
   System.out.println("3. Tampilkan Antrian");
   System.out.println("4. Reverse");
   System.out.println("5. Keluar");

   Digunakan untuk menampilkan daftar menu program kepada pengguna.
   Menu hanya ditampilkan sekali di awal program.

8)
```
while (pilihan_2002 != 5){
System.out.print("\nPilih menu: ");
pilihan_2002=input.nextInt();
input.nextLine();
```

   Digunakan untuk mengulang program selama pengguhna belum *input 5*. Selanjutnya adalah menerima input dari *user*.

9) *if (pilihan_2002 ==1) {*
*System.**out**.print("Masukkan Nama Pelanggan : ");*
String nama=input.nextLine();
antrian.enqueue_2002(nama);

}else if (pilihan_2002 == 2) {
antrian.dequeue_2002();

}else if (pilihan_2002 == 3) {
antrian.tampil_2002();

}else if (pilihan_2002==4) {
antrian.reverse();
antrian.tampil_2002();

}else if (pilihan_2002==5) {
*System.**out**.println("Program Selesai!");*

}else {
*System.**out**.println("Pilihan Tidak Valid!");*
}

Digunakan menampilkan hasil yang sesuai berdasarkan *input* dari *user* dan jika *user* memberikan *input* yang tidak valid maka akan memunculkan pesan "Pilihan tidak valid".

10) *} input.close();*

Digunakan untuk menutup objek Scanner setelah program selesai dijalankan.

2. Hasil

```
Console
<terminated> AntrianLoket_2511532002 [Java Application] C:\Users\user\
=== PROGRAM ANTRIAN LOKET ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

Pilih menu: 1
Masukkan Nama Pelanggan : Haliya
Data berhasil ditambahkan ke antrian

Pilih menu: 1
Masukkan Nama Pelanggan : Manon
Data berhasil ditambahkan ke antrian

Pilih menu: 3
Isi Antrian :
1. Haliya
2. Manon

Pilih menu: 4
Antrian berhasil dibalik!
Isi Antrian :
1. Manon
2. Haliya

Pilih menu: 2
Manon telah dilayani.

Pilih menu: 3
Isi Antrian :
1. Haliya

Pilih menu: 5
Program Selesai!
```

3. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, program Simulasi Antrian Loket berhasil dibuat dengan menggunakan struktur data *Queue* yang menerapkan konsep *FIFO (First In, First Out).* Program ini mampu menjalankan seluruh fitur yang dirancang, yaitu menambahkan pelanggan ke dalam antrian (*enqueue*), menghapus pelanggan yang telah dilayani (*dequeue*), menampilkan isi antrian, mengecek kondisi antrian (kosong atau penuh), serta membalik urutan antrian (*reverse*). Dengan demikian, program ini dapat merepresentasikan sistem antrian secara sederhana sesuai dengan konsep dasar struktur data *Queue*.